

Informe Final del Compilador de HULK

Frontend, análisis semántico, representación intermedia, backend LLVM y runtime

Camilo Pérez Fleita
Guillermo Hugues Cardona
Mauricio Medina Hernández

Proyecto de Compilación / Lenguajes de Programación

20 de junio de 2026

Resumen

Este informe documenta el diseño, la implementación y el estado alcanzado por nuestro compilador de HULK, desarrollado en el repositorio `mauricio04mh/hulk-compiler`. El compilador está organizado en ocho crates Rust: generadores propios de lexer y parser, un frontend que produce AST, una fase semántica con inferencia iterativa e HIR tipada, un lowering a representación intermedia propia (Hulk IR), un backend de emisión de LLVM IR, y un runtime en C. El documento explica con detalle las estructuras de datos centrales de cada módulo, las decisiones de arquitectura tomadas por el equipo, el modelo de tipos en la IR, la organización de objetos y vtables en el runtime, y la integración del driver con `clang`. También se documentan las nueve extensiones implementadas sobre el núcleo del lenguaje: inferencia iterativa de tipos, protocolos e interfaces estructurales, arreglos y vectores indexados, generadores e iterables de usuario, lambdas y closures, la forma `define`, el bucle `for` sobre iterables, visibilidad protegida de atributos en jerarquías de herencia, y pattern matching con verificación de exhaustividad en compilación.

Índice

1. Introducción	6
2. Objetivos y alcance	6
2.1. Objetivo general	6
2.2. Objetivos específicos	6
2.3. Resumen de estado	7

3. Arquitectura del repositorio	7
3.1. Workspace Rust	7
3.2. Pipeline de compilación	8
3.3. Ventajas y costos de la separación por fases	8
4. Frontend léxico y sintáctico	9
4.1. Crate <code>hulk-lexgen</code> : especificación declarativa de lexer	9
4.2. Crate <code>hulk-parsegen</code> : parser LL(1) y Pratt parser	10
4.3. Crate <code>hulk-frontend</code> : AST y pipeline de parseo	11
5. Análisis semántico e HIR tipada	12
5.1. Crate <code>hulk-sema</code> : responsabilidades generales	12
5.2. Sistema de tipos: <code>Type</code> y <code>TypeRegistry</code>	12
5.3. Scopes y entorno de tipos: <code>TypeEnv</code>	13
5.4. Inferencia iterativa de tipos	13
5.5. HIR tipada: estructura y contratos	14
5.6. Errores semánticos	15
6. Representación intermedia propia (Hulk IR)	16
6.1. Motivación y diseño general	16
6.2. Tipos e identificadores con tipos fuertes	16
6.3. Layouts de tipos: atributos y <code>vtable</code>	17
6.4. Funciones: clasificación por <code>kind</code>	17
6.5. Valores y lugares: la distinción <code>read/write</code>	18
6.6. Conjunto de instrucciones	18
7. Lowering: de HIR a Hulk IR	19
7.1. Responsabilidades del crate <code>hulk-lower</code>	19
7.2. Construcción de layouts de tipos y <code>vtables</code>	20
7.3. Lowering de expresiones	20
7.4. Lowering de lambdas y closures	21
8. Backend LLVM y runtime	21

8.1. Crate <code>hulk-codegen-llvm</code> : emisión de LLVM IR	21
8.2. Mapeo de tipos IR a LLVM	22
8.3. ABI con el runtime	23
8.4. Crate <code>runtime/</code> : estructuras y modelo de objetos en C	24
8.5. Builtins matemáticos y métodos de String	25
8.6. Comprobaciones de seguridad en runtime	26
8.7. Driver e integración con <code>clang</code>	27
8.8. Makefile	27
9. Estrategia de pruebas	27
9.1. Pruebas unitarias por capa	27
9.2. Suite de programas HULK de extremo a extremo	29
9.3. CI	30
10. Extensiones del lenguaje	31
10.1. Extensión 1: inferencia de tipos sin anotaciones	31
10.1.1. Motivación	31
10.1.2. Cómo se implementó	31
10.1.3. Por qué mejora el sistema	32
10.2. Extensión 2: protocolos e interfaces estructurales	32
10.2.1. Motivación	32
10.2.2. Cómo se implementó	33
10.2.3. Por qué mejora el sistema	33
10.3. Extensión 3: arreglos indexados con inicialización funcional	34
10.3.1. Motivación	34
10.3.2. Tres formas de creación	34
10.3.3. Cómo se implementó	34
10.3.4. Por qué mejora el sistema	35
10.4. Extensión 4: generadores e iterables de usuario	35
10.4.1. Motivación	35
10.4.2. Cómo se implementó	36
10.4.3. Por qué mejora el sistema	36

10.5. Extensión 5: lambdas, closures y funciones de orden superior	36
10.5.1. Motivación	36
10.5.2. Cómo se implementó	37
10.5.3. Por qué mejora el sistema	37
10.6. Extensión 6: define como definición de expresión única	38
10.6.1. Motivación	38
10.6.2. Cómo se implementó	38
10.6.3. Por qué mejora el sistema	39
10.7. Extensión 7: for sobre iterables y el builtin range	39
10.7.1. Motivación	39
10.7.2. Comportamiento de range	39
10.7.3. Cómo se implementó	39
10.7.4. Por qué mejora el sistema	40
10.8. Extensión 8: visibilidad protegida de atributos en jerarquías de herencia . . .	40
10.8.1. Contexto: lo que dice la especificación	40
10.8.2. Qué se implementó: visibilidad protegida	40
10.8.3. Evidencia directa en los tests	41
10.8.4. Cómo funciona en el compilador: la lógica exacta	42
10.8.5. Justificaciones de por qué esta extensión mejora el lenguaje	43
10.8.6. Regla final aplicada por el compilador	45
10.9. Extensión 9: pattern matching	45
10.9.1. Motivación	45
10.9.2. Sintaxis	46
10.9.3. Tipos de patrones	46
10.9.4. Verificación de exhaustividad	46
10.9.5. Cómo se implementó	47
10.9.6. Por qué mejora el sistema	47
11.Comparación con la planificación inicial	48
12.Limitaciones	49
12.1. Runtime sin recolector de basura	49

12.2. Cobertura del backend	49
12.3. Macros	49
13.Instrucciones de uso	49
13.1. Compilación del proyecto	49
13.2. Uso del compilador	50
13.3. Requisitos	50
14.Conclusiones	50
A. Checklist final de entrega	50
B. Programa demostrativo sugerido	51

1. Introducción

Construir un compilador para HULK requiere preservar el significado del programa a través de múltiples capas de representación. El objetivo fue construir un pipeline defendible, inspeccionable y probado por capas: desde el reconocimiento léxico hasta la generación de LLVM IR enlazable con un runtime en C.

La arquitectura adoptada es estrictamente por fases. El frontend reconoce tokens y estructura sintáctica; el AST elimina detalles de la gramática concreta; la fase semántica resuelve nombres, scopes, tipos, atributos, métodos, `self`, `base` y builtins; la HIR conserva la estructura de alto nivel ya tipada; el lowering transforma esa HIR en una representación intermedia propia (Hulk IR); y el backend emite LLVM IR enlazable con el runtime.

Este informe distingue tres niveles de soporte:

1. **Soporte sintáctico:** la construcción se reconoce y se representa en AST.
2. **Soporte semántico e intermedio:** la construcción se valida y se baja a HIR/IR.
3. **Soporte ejecutable:** el backend LLVM y el runtime permiten producir un binario funcional.

2. Objetivos y alcance

2.1. Objetivo general

Implementar un compilador modular para HULK capaz de leer programas fuente, analizarlos semánticamente, traducirlos a una representación intermedia propia y generar LLVM IR enlazable con un runtime en C.

2.2. Objetivos específicos

1. Implementar un frontend reutilizable a partir de especificaciones léxicas y gramaticales declarativas.
2. Construir un AST expresivo para declaraciones, expresiones, tipos, protocolos, funciones, métodos, vectores, lambdas y operaciones de objeto.
3. Resolver nombres y scopes, incluyendo variables locales, parámetros, builtins, funciones globales, atributos, métodos, `self` y `base`.
4. Implementar chequeo e inferencia de tipos sobre primitivos, tipos de usuario, protocolos, vectores, iterables y functors.
5. Producir una HIR tipada como contrato entre semántica y lowering.

6. Diseñar una IR propia de tres direcciones, con secciones de tipos, datos y código.
7. Emitir LLVM IR para el subconjunto ejecutable y delegar operaciones complejas al runtime.
8. Ofrecer una interfaz de línea de comandos para compilar y depurar cada fase.

2.3. Resumen de estado

Requisito	Evidencia	Estado	Comentario técnico
Lenguaje de implementación	Workspace Rust + runtime C	Cumplido	Compilador en Rust; soporte de ejecución en C.
Frontend	<code>hulk-lexgen</code> , <code>hulk-parsegen</code> , <code>hulk-frontend</code>	Cumplido	Lexer, parser LL(1), Pratt parser y AST builder.
Semántica	<code>hulk-sema</code>	Cumplido amplio	TypeRegistry, scopes, HIR tipada, inferencia iterativa.
IR propia	<code>hulk-ir</code> , <code>hulk-lower</code>	Cumplido	IR con secciones <code>.TYPES</code> , <code>.DATA</code> , <code>.CODE</code> .
Backend LLVM	<code>hulk-codegen-llvm</code>	Funcional completo	Emite LLVM IR; ABI completo con el runtime; guards de división/-módulo por cero.
Runtime	<code>runtime/hulk_runtime.c</code>	Funcional con safety	Strings, objetos, vtables, vectores, rangos, closures; arena 64MB; comprobaciones de bounds, nulos y vtable.
CLI	<code>hulk-driver</code>	Cumplido	Compila a <code>./output</code> ; flags para depurar por fase.
CI	GitHub Actions	Cumplido	<code>cargo test -workspace</code> pasa en CI.

3. Arquitectura del repositorio

3.1. Workspace Rust

La raíz del proyecto define un workspace con ocho crates principales más una carpeta de runtime en C. Esta división evita un compilador monolítico y permite probar cada fase de forma independiente.

Crate	Responsabilidad
<code>hulk-lexgen</code>	Generación de lexer a partir de especificaciones <code>LexerSpec</code> .
<code>hulk-parsegen</code>	Generación de parser, cálculo FIRST/FOLLOW, tabla LL(1) y Pratt parser.
<code>hulk-frontend</code>	API pública de parseo, pipeline cacheado y construcción de AST.
<code>hulk-sema</code>	Resolución de nombres, <code>TypeRegistry</code> , chequeo e inferencia semántica, HIR tipada.
<code>hulk-ir</code>	Modelo de IR: tipos, datos, funciones, instrucciones, valores e impresión textual.
<code>hulk-lower</code>	Traducción de <code>SemanticProgram</code> /HIR a <code>IrProgram</code> .
<code>hulk-codegen-llvm</code>	Emisión textual de LLVM IR y ABI de llamadas al runtime.
<code>hulk-driver</code> <code>runtime/</code>	Binario <code>hulkc</code> : integra todas las fases y llama a <code>clang</code> . Biblioteca C con objetos, vtables, vectores, rangos y closures.

3.2. Pipeline de compilación

Listing 1: Pipeline completo del compilador

```

archivo .hulk
-> hulk-lexgen (LexerSpec -> tokens)
-> hulk-parsegen (LL(1) + PrattParser -> CstNode)
-> hulk-frontend (AstBuilder -> Program)
-> hulk-sema (analyze_program -> SemanticProgram + TypeRegistry + HIR)
-> hulk-lower (lower_program -> IrProgram)
-> hulk-codegen-llvm (emit_llvm -> LLVM IR textual)
-> clang output.ll runtime/hulk_runtime.c -lm
-> ejecutable ./output

```

El driver (`hulk-driver/src/main.rs`) orquesta todo el pipeline. Sin flags, ejecuta cada fase hasta producir `./output`. Con flags como `-ast`, `-hir`, `-ir` o `-emit-llvm`, detiene el pipeline en la fase solicitada e imprime la representación intermedia. Esta separación es central para la depuración: el compilador no es una caja negra.

3.3. Ventajas y costos de la separación por fases

La arquitectura tiene tres ventajas concretas:

1. **Contratos claros.** El lowering consume HIR tipada (`SemanticProgram`) y no repite validaciones de alto nivel. El backend LLVM consume `IrProgram` sin conocer la

semántica.

2. **Pruebas por capa.** Cada crate tiene sus propias pruebas Rust. El crate `hulk-codegen-llvm` incluye pruebas de integración que compilan con `clang` y verifican la salida estándar.
3. **Crecimiento incremental.** Una nueva construcción puede entrar primero en AST/HIR/IR antes de que el backend la soporte completamente.

El costo es la necesidad de mantener representaciones sincronizadas. Agregar arreglos, por ejemplo, requirió cambios en lexer, parser, AST, semántica, IR, lowering, backend y runtime.

4. Frontend léxico y sintáctico

4.1. Crate `hulk-lexgen`: especificación declarativa de lexer

El lexer de HULK no se codifica manualmente. En su lugar, se define una estructura `LexerSpec` (en `hulk-lexgen/src/spec/lexer_spec.rs`) que el crate convierte en un lexer funcional:

Listing 2: Estructura central de especificación léxica

```
pub struct LexerSpec {
    pub exact_rules: Vec<ExactRule>,    // palabras clave y operadores fijos
    pub identifier: Option<IdentifierRule>,
    pub number: Option<NumberRule>,
    pub string: Option<StringRule>,
    pub skip_whitespace: bool,
    pub line_comment: Option<LineCommentRule>,
}
```

Cada tipo de regla captura solo los atributos necesarios. `ExactRule` registra el texto exacto, el nombre del token, si es palabra clave y la prioridad. `NumberRule` indica si se aceptan enteros, flotantes o ambos. `StringRule` configura el delimitador y las secuencias de escape habilitadas. `LineCommentRule` especifica el prefijo del comentario.

Esta decisión de diseño tiene una consecuencia importante: el lexer es reutilizable para cualquier lenguaje con una especificación similar. Para HULK en particular, la especificación registra palabras clave como `function`, `define`, `type`, `inherits`, `new`, `self`, `base`, `let`, `in`, `while`, `for`, `protocol`, `interface`, `extends`; operadores compuestos como `@@`, `@`, `==`, `!=`, `<=`, `:=`, `=>`, `->`; y construcciones de literales para números, strings e identificadores.

Las posiciones léxicas (línea y columna) se conservan en cada token, lo que permite que el pipeline emita mensajes de error ubicados.

4.2. Crate hulk-parsegen: parser LL(1) y Pratt parser

El crate `hulk-parsegen` combina dos estrategias de parseo. El parser LL(1) se genera a partir de una especificación de gramática y usa una tabla FIRST/FOLLOW para decidir qué producción expandir. Este parser maneja la estructura global del programa: declaraciones de funciones, tipos y protocolos, así como los terminadores de declaración.

Para las expresiones se usa un Pratt parser (en `hulk-parsegen/src/runtime/pratt.rs`). Esta decisión es adecuada porque las expresiones de HULK tienen múltiples niveles de precedencia, operadores prefijos, sufijos de llamada, acceso por punto, indexación, type test, type cast y lambdas; representar todo eso en una gramática LL(1) obligaría a añadir decenas de no-terminales intermedios.

La configuración del Pratt parser se declara en `PrattConfig`:

Listing 3: Configuración del Pratt parser

```
pub struct PrattConfig {
    pub binary_ops: HashMap<String, OperatorInfo>, // precedencia + asociatividad
    pub unary_prefix_ops: HashSet<String>,
    pub primary_tokens: HashSet<String>,
    pub lparen: String, pub rparen: String,
    pub new_kw: Option<String>,
    pub self_kw: Option<String>,
    pub base_kw: Option<String>,
    pub dot: Option<String>,
    pub is_kw: Option<String>,
    pub as_kw: Option<String>,
    pub lbracket: Option<String>, pub rbracket: Option<String>,
    pub arrow: Option<String>, pub funcarrow: Option<String>,
    pub if_kw: ..., pub while_kw: ..., pub for_kw: ..., // control-flow como
    expresiones
    pub lbrace: ..., pub semicolon: ..., pub let_kw: ...,
    pub match_kw: Option<String>, pub wildcard: Option<String>, // pattern matching
}
```

Los niveles de precedencia reales de HULK están registrados en la tabla de operadores binarios:

Precedencia	Operadores	Asoc.	Ejemplo
1	(or)	Izquierda	a b
2	& (and)	Izquierda	a & b
3	==, !=	Izquierda	a == b
4	<, <=, >, >=	Izquierda	x <5
5	@, @@	Izquierda	.a"@ "b"
6	+, -	Izquierda	x + 1
7	*, /, %	Izquierda	x * 2

El método central es `parse_expression_bp(min_bp)`, que implementa el algoritmo de Pratt: parsea un prefijo, luego itera consumiendo operadores infijos o sufijos siempre que su precedencia sea mayor o igual a `min_bp`. Para operadores asociativos por la derecha (como `^`) el umbral del lado derecho es igual al nivel actual en lugar de uno más.

Los sufijos se manejan con prioridad absoluta sobre operadores binarios, sin entrar en la tabla de precedencias: las llamadas `f(args)`, el acceso por punto `obj.method()`, la indexación `v[i]`, y los operadores `is/as` se procesan como sufijos dentro del bucle principal de `parse_expression_bp_impl`. Esto refleja que en HULK el acceso a miembros y la indexación tienen mayor precedencia que cualquier operador binario.

El Pratt parser también maneja recursión controlada a través de un contador de profundidad con límite `MAX_PARSE_DEPTH = 512`, lo que evita desbordamientos de pila con expresiones muy anidadas.

4.3. Crate `hulk-frontend`: AST y pipeline de parseo

El `crate hulk-frontend` expone la API pública de parseo. La función `parse_hulk_types_program(source)` es el punto de entrada que usan todos los demás crates. Internamente invoca el pipeline completo: `lexer → tokens → parser LL(1) → CST → AstBuilder → Program`.

La separación entre CST (árbol de sintaxis concreta, manejado por `hulk-parsegen`) y AST (árbol de sintaxis abstracta, construido por `AstBuilder` en `hulk-frontend/src/builder.rs`) es una decisión deliberada. El CST refleja la gramática y puede contener nodos auxiliares como `LetBindingTail`, `ElifList` o `GroupExpr`. El AST elimina esos detalles y produce una estructura limpia para las fases posteriores.

El AST principal está definido en `hulk-frontend/src/ast.rs`:

Listing 4: Estructura raíz del AST

```
pub struct Program {
    pub declarations: Vec<Decl>,
    pub entry: Expr,
}

pub enum Decl {
    Function(FunctionDecl),
    Type(TypeDecl),
    Protocol(ProtocolDecl),
}
```

Las expresiones del AST cubren todos los constructos del lenguaje: `Number`, `String`, `Bool`, `Var`, `Unary`, `Binary`, `Assign`, `Let`, `Block`, `If`, `While`, `For`, `Call`, `MethodCall`, `New`, `MemberAccess`, `SelfRef`, `BaseCall`, `TypeTest`, `TypeCast`, `VectorLiteral`, `VectorGenerator`, `NewVector`,

`VectorIndex`, `Lambda`, `Match`. Cada nodo incluye un `Span` con información de posición para diagnósticos. El nodo `Match` agrupa el escrutinio y una lista de `MatchArm`, cada uno con un `Pattern` (`Wildcard`, `Literal`, `Binding` o `TypePattern`) y el cuerpo del brazo.

La distinción `FunctionDecl::is_macro` marca si la declaración usa `define` en lugar de `function`. La fase semántica usa esta distinción para clasificar el tipo de declaración.

El driver distingue errores léxicos de sintácticos: si el mensaje contiene “lex error”, “Un-terminated” o “unexpected character”, sale con código 1; si es un error de gramática, sale con código 2. Esto permite a los evaluadores automáticos categorizar los errores por fase.

5. Análisis semántico e HIR tipada

5.1. Crate `hulk-sema`: responsabilidades generales

El crate `hulk-sema` es el módulo central del compilador. Recibe un `Program` (AST) y produce un `SemanticProgram` que contiene tres elementos:

Listing 5: Contrato de salida de la fase semántica

```
pub struct SemanticProgram {
    pub hir: HirProgram,           // HIR tipada lista para lowering
    pub registry: TypeRegistry,    // registro completo de tipos y protocolos
    pub functions: HashMap<String, FunctionType>, // firmas resueltas
}
```

Las tareas principales son: registrar funciones, tipos, protocolos y builtins; construir el `TypeRegistry`; resolver scopes y símbolos; validar aridad y tipos en llamadas; resolver `self`, `base`, atributos y despacho de métodos; verificar herencia, ciclos y overrides; comprobar conformidad de protocolos; inferir tipos cuando faltan anotaciones; y producir HIR tipada.

5.2. Sistema de tipos: `Type` y `TypeRegistry`

El sistema de tipos internos usa una enumeración `Type` con variantes: `Number`, `String`, `Boolean`, `Object`, `User(String)`, `Vector(Box<Type>)`, `Iterable(Box<Type>)`, `Functor`, y `Unknown`. El tipo `Unknown` actúa como centinela de error: permite que el chequeo continúe después de encontrar un error sin propagar cascadas de diagnósticos falsos.

El `TypeRegistry` (en `hulk-sema/src/context.rs`) almacena toda la información de tipos de usuario: nombre, parámetros de constructor, padre, atributos, métodos y protocolos que conforman. Su construcción ocurre en varias pasadas:

1. Primera pasada: se recogen los nombres de todos los tipos y protocolos para permitir referencias adelantadas.
2. Segunda pasada: se rellenan atributos, métodos, firmas y relaciones de herencia.

3. Tercera pasada: se propagan parámetros de constructor en herencia pasante (cuando un hijo no declara sus propios parámetros).
4. Cuarta pasada: se registra `Iterable` como builtin si el usuario no lo definió explícitamente.

La validación de herencia comprueba: padres inexistentes, herencia desde primitivos (`Number`, `String`, `Boolean`), ciclos de herencia mediante detección de caminos, y que los overrides de métodos preserven la firma del padre.

5.3. Scopes y entorno de tipos: TypeEnv

El chequeo de tipos usa una estructura `TypeEnv` (definida en `hulk-sema/src/checker.rs`) que mantiene una pila de scopes:

Listing 6: Entorno de tipos con pila de scopes

```
pub struct TypeEnv {
    scopes: Vec<HashMap<String, Type>>, // pila de entornos léxicos
    pub functions: HashMap<String, FunctionType>,
    pub registry: TypeRegistry,
    pub current_type: Option<String>, // tipo activo (para self)
    pub current_type_parent: Option<String>, // padre del tipo activo (para base)
    pub current_method: Option<String>, // método activo (para base())
    errors: Vec<SemanticError>,
}
```

La resolución de variables (`resolve_var`) busca hacia arriba en la pila de scopes (`iter().rev()`), lo que implementa el shadowing léxico correctamente. Al entrar a un bloque `let` o al cuerpo de una función se llama a `push_scope()`; al salir se llama a `pop_scope()`.

La resolución de atributos (`self.attr`) usa `current_type` para buscar el atributo en el `TypeRegistry`. La resolución de llamadas a `base()` usa `current_type_parent` y `current_method` para localizar exactamente el método del padre correspondiente.

5.4. Inferencia iterativa de tipos

Una decisión clave de diseño fue implementar inferencia iterativa en el `HirBuilder` (`hulk-sema/src/hir_builder.rs`, el módulo más grande con más de 2800 líneas). En lugar de ejecutar una sola pasada de chequeo, el `HirBuilder` itera:

1. Ejecuta una pasada de análisis sobre funciones, métodos y constructores.
2. Si alguna firma cambió respecto a la pasada anterior, repite.
3. Solo cuando no hay cambios ejecuta una pasada final que reporta errores de parámetros no resueltos.

Este diseño permite refinar firmas parciales. Si una función usa un parámetro en una suma numérica, el compilador puede inferir que es `Number`. Si una llamada pasa un argumento concreto a una función con firma parcial, la firma se refina desde el sitio de uso. Esto soporta casos como:

Listing 7: Inferencia desde el cuerpo

```
function double(x) => x + x; // x inferido como Number por la suma
print(double(21));          // imprime 42
```

Listing 8: Inferencia desde el sitio de llamada

```
function id(x) => x;          // x inferido como String desde la llamada
print(id("hello"));          // imprime "hello"
```

5.5. HIR tipada: estructura y contratos

La HIR está definida en `hulk-sema/src/hir.rs`. Es el contrato entre la semántica y el lowering. Cada expresión HIR tiene cuatro campos:

Listing 9: Nodo raíz de la HIR

```
pub struct HirExpr {
    pub id: HirId,      // identificador único (u32)
    pub span: Span,     // posición en el fuente
    pub ty: Type,       // tipo resuelto por la semántica
    pub kind: HirExprKind,
}
```

Los identificadores de símbolos (`SymbolId(u32)`) resuelven la ambigüedad entre variables con el mismo nombre en diferentes scopes. Cada binding de variable tiene su propio `SymbolId`; las referencias a esa variable usan el mismo ID, lo que permite al lowering generar nombres de locales únicos.

Las llamadas a funciones se clasifican en tres variantes de `HirCallee`:

Listing 10: Clasificación de callees en HIR

```
pub enum HirCallee {
    Builtin { name: String, signature: FunctionType },
    GlobalFunction { name: String, signature: FunctionType },
    LocalFunctor { name: String, symbol: SymbolId, signature: FunctionType },
}
```

Las llamadas a métodos usan `DispatchKind` para distinguir el tipo de despacho:

Listing 11: Tipos de despacho de métodos

```
pub enum DispatchKind {
    Virtual {
```

```

        receiver_static_type: Type,
        method_name: String,
        signature: FunctionType,
    },
    Static {
        function_label: String,
        signature: FunctionType,
    },
    Base {
        parent_type: String,
        method_name: String,
        signature: FunctionType,
    },
}

```

El despacho `Virtual` se usa cuando el tipo estático del receptor es un tipo de usuario o protocolo (se resuelve en runtime via `vtable`). El despacho `Static` ocurre cuando el tipo es concreto y el método no puede ser sobrescrito. El despacho `Base` es específico para llamadas `base()` dentro de métodos sobrescritos.

Los accesos a atributos usan `ResolvedMember::Attribute` que guarda el tipo propietario (`owner_type`) y el nombre del atributo, lo que el lowering usa para generar el ID de atributo correcto según el layout del tipo.

5.6. Errores semánticos

La enumeración `SemanticError` (en `hulk-sema/src/error.rs`) cubre más de 25 casos: funciones duplicadas, parámetros duplicados, símbolos indefinidos, métodos indefinidos, aridad incorrecta, tipos desconocidos, incompatibilidades de tipo, operandos inválidos, condiciones no booleanas, inferencia fallida, herencia circular, métodos faltantes de protocolos, acceso a atributos privados, herencia desde primitivos, firmas incompatibles en override y `NonExhaustiveMatch` (match sin brazo wildcard o binding final).

Las tres variantes más frecuentes para el usuario (`UndefinedVariable`, `UndefinedFunction` y `UndefinedMethod`) llevan un campo `Span { line, col }` que se propaga desde el nodo AST correspondiente. Cuando el span es no-nulo el mensaje incluye la ubicación:

Listing 12: Ejemplos de errores semánticos con ubicación

```

SEMANTIC error: Undefined variable 'x' at line 3:9
SEMANTIC error: Undefined function 'foo' at line 7:1
SEMANTIC error: Undefined method 'bar' for type 'MyType' at line 12:5

```

Esta variedad de errores es importante para la evaluación: el compilador no solo acepta programas correctos, sino que rechaza programas inválidos con diagnósticos diferenciados por categoría.

6. Representación intermedia propia (Hulk IR)

6.1. Motivación y diseño general

La Hulk IR actúa como puente entre la HIR semántica y el backend LLVM. Su función es hacer explícitos los valores intermedios, el control de flujo, los layouts lógicos de objetos y las operaciones de runtime, sin depender de la sintaxis de alto nivel ni comprometerse con los detalles de memoria de LLVM.

La IR está definida en `hulk-ir/src/lib.rs`. Un `IrProgram` contiene cuatro componentes:

Listing 13: Estructura de `IrProgram`

```
pub struct IrProgram {
    pub types: Vec<IrType>,           // layouts lógicos de tipos de usuario
    pub data: Vec<IrData>,           // datos estáticos (principalmente strings)
    pub functions: Vec<IrFunction>, // funciones, métodos, lambdas y entry
    pub entry: FunctionId,           // función de entrada del programa
}
```

Textualmente se imprime en tres secciones: `.TYPES`, `.DATA` y `.CODE`, lo que facilita la inspección manual y las golden tests.

6.2. Tipos e identificadores con tipos fuertes

Toda referencia a un elemento de la IR usa un identificador con tipo fuerte (un `struct newtype` sobre `u32`): `TypeId`, `AttrId`, `MethodSlot`, `DataId`, `FunctionId`, `ParamId`, `LocalId`, `TempId`, `LabelId`. Esta decisión evita confundir accidentalmente, por ejemplo, un `LocalId` con un `ParamId` en el código del lowering.

Las referencias de tipo en instrucciones usan `IrTypeRef`:

Listing 14: Sistema de tipos en la IR

```
pub enum IrTypeRef {
    Number,
    String,
    Boolean,
    Object,
    User(String),
    Vector(Box<IrTypeRef>),
    Iterable(Box<IrTypeRef>),
    Functor {
        capture_types: Vec<IrTypeRef>,
        params: Vec<IrTypeRef>,
        ret: Box<IrTypeRef>,
    },
    Unknown,
```



```
}
```

La variante `Functor` incluye `capture_types` además de los parámetros y el retorno. Esta información extra permite al backend LLVM generar la estructura de closure correcta con el número exacto de capturas.

6.3. Layouts de tipos: atributos y vtable

Un `IrType` almacena el layout lógico de un tipo de usuario:

Listing 15: Layout lógico de un tipo en IR

```
pub struct IrType {  
    pub id: TypeId,  
    pub name: String,  
    pub parent: Option<String>,  
    pub attributes: Vec<IrAttribute>, // cada uno con AttrId, nombre y tipo  
    pub methods: Vec<IrMethod>,      // cada uno con MethodSlot, nombre y función  
}
```

Los métodos se referencian por `MethodSlot`, que es el índice dentro de la vtable. Si un tipo hijo sobrescribe un método del padre, usa el mismo `MethodSlot` pero apunta a una función diferente. Esto es exactamente lo que el backend necesita para construir vtables correctas.

6.4. Funciones: clasificación por kind

Cada `IrFunction` tiene un campo `kind` que distingue cuatro categorías:

Listing 16: Tipos de función en IR

```
pub enum IrFunctionKind {  
    Entry, // función de entrada del programa  
    Function, // función global  
    Method { owner_type: String, method_name: String }, // método de tipo  
    Lambda, // lambda capturada  
}
```

Esta distinción permite al backend tratar cada categoría apropiadamente: las funciones de método reciben `self` como primer parámetro implícito; las lambdas reciben la closure como contexto adicional.

Una función IR declara por separado sus parámetros (`Vec<IrParam>`), locales (`Vec<IrLocal>`), y temporales (`Vec<IrTemp>`). Los temporales son generados automáticamente por el lowering para resultados intermedios de expresiones. Los locales corresponden a variables nombradas (`let`, parámetros de `for`, variables de captura). Esta separación entre temporales y locales refleja la distinción usual en IR de tres direcciones.

6.5. Valores y lugares: la distinción read/write

La IR distingue entre `IrValue` (lo que se puede leer) e `IrPlace` (donde se puede escribir):

Listing 17: Valores y lugares en IR

```
pub enum IrValue {
    Temp(TempId), Local(LocalId), Param(ParamId),
    ConstNumber(f64), ConstBool(bool),
    DataRef(DataId), // referencia a string estático
    Null, Unit,
}

pub enum IrPlace {
    Temp(TempId),
    Local(LocalId),
}
```

Solo los temporales y locales son escribibles (`IrPlace`). Los parámetros son de solo lectura (`IrValue` pero no `IrPlace`). Las constantes y referencias a datos tampoco son escribibles. Esta distinción hace que las asignaciones ilegales sean imposibles de expresar en la IR.

6.6. Conjunto de instrucciones

El conjunto de instrucciones cubre todas las operaciones necesarias del lenguaje:

Instrucción	Descripción
<code>Assign{dst, src}</code>	Copia de valor.
<code>Unary{dst, op, value}</code>	Operación unaria (<code>Not</code> , <code>Neg</code>).
<code>Binary{dst, op, l, r}</code>	Operación binaria (16 operadores: aritméticos, comparación, lógicos, concatenación).
<code>Jump(label)</code>	Salto incondicional.
<code>Branch{cond, then, else}</code>	Salto condicional.
<code>Label(id)</code>	Destino de salto.
<code>Return(Option<IrValue>)</code>	Retorno de función.
<code>Call{dst, function, args}</code>	Llamada a función por nombre.
<code>StaticCall{...}</code>	Llamada estática a método (sin búsqueda vtable).
<code>VirtualCall{receiver, static_type, method, slot, ...}</code>	Llamada dinámica por slot de vtable.
<code>BaseCall{parent_type, method, ...}</code>	Llamada al método del padre (<code>base()</code>).

<code>Allocate{dst, type_name}</code>	Asignación de objeto en heap.
<code>GetAttr{dst, object, attr}</code>	Lectura de atributo por <code>AttrId</code> .
<code>SetAttr{object, attr, value}</code>	Escritura de atributo por <code>AttrId</code> .
<code>NewVector{dst, elements}</code>	Creación de vector con elementos iniciales.
<code>VectorLen{dst, vector}</code>	Longitud del vector.
<code>VectorPush{vector, value}</code>	Inserción al final del vector.
<code>VectorGet{dst, vector, idx}</code>	Indexación del vector.
<code>VectorSet{vector, idx, value}</code>	Asignación por índice.
<code>MakeClosure{dst, function, captures}</code>	Construcción de closure con capturas.
<code>ClosureCall{dst, closure, args}</code>	Invocación de closure.
<code>GetCapture{dst, closure, idx, ty}</code>	Lectura de variable capturada.
<code>TypeTest{dst, value, type_name}</code>	Operador <code>is</code> .
<code>TypeCast{dst, value, type_name}</code>	Operador <code>as</code> .

La instrucción `VirtualCall` incluye `receiver_static_type` además del slot. Esto permite al backend LLVM buscar la vtable del tipo estático del receptor para localizar el slot, en lugar de necesitar un mecanismo de despacho más complejo.

La instrucción `GetCapture` se añadió explícitamente para separar la lectura de variables capturadas del mecanismo general de despacho. Un acceso a captura dentro de una lambda no es lo mismo que un acceso a local o parámetro; requiere desreferenciar la estructura de closure.

7. Lowering: de HIR a Hulk IR

7.1. Responsabilidades del crate `hulk-lower`

El módulo `hulk-lower/src/lib.rs` (1517 líneas) transforma el `SemanticProgram` en un `IrProgram`. No realiza validación semántica; la HIR ya es correcta. Su función es:

1. Crear los layouts de tipos (**IrType**) con atributos heredados y slots de método.
2. Generar funciones IR para cada función global, método y lambda.
3. Traducir expresiones HIR a secuencias de instrucciones IR con temporales explícitos.
4. Resolver llamadas de método a **VirtualCall**, **StaticCall** o **BaseCall** según el **DispatchKind**.
5. Manejar vectores, rangos, closures y generadores.

7.2. Construcción de layouts de tipos y vtables

Al procesar un **HirTypeDecl**, el lowering:

1. Hereda los atributos del padre (si existe) con sus **AttrId** originales.
2. Añade los atributos propios con IDs consecutivos a partir del máximo del padre.
3. Hereda los slots de métodos del padre.
4. Para cada método propio: si el nombre coincide con un slot heredado (override), actualiza la función destino del slot existente; si es nuevo, añade un slot con el siguiente índice disponible.

Esta lógica de herencia de slots garantiza que las vtables de los hijos tengan el mismo layout que la del padre para los métodos heredados, lo cual es el requisito fundamental del despacho dinámico polimórfico.

7.3. Lowering de expresiones

El lowering de expresiones sigue un patrón uniforme: cada expresión HIR se traduce a una secuencia de instrucciones que deja el resultado en un nuevo temporal. El lowering acumula instrucciones en un buffer mutable y devuelve el **IrValue** que contiene el resultado.

Para expresiones complejas como bloques, el lowering procesa cada subexpresión en orden y descarta los resultados intermedios, conservando solo el resultado de la última expresión del bloque. Para **if/elif/else**, genera etiquetas y saltos condicionales:

Listing 18: Patrón de lowering para if-else

```
branch %cond ? L_then : L_else
label L_then
... codigo then ...
%t_result = %t_then
jump L_end
label L_else
... codigo else ...
```

```
%t_result = %t_else
label L_end
return %t_result
```

Para `while`, genera un bucle con etiqueta de comprobación y etiqueta de salida. Para `for`, invoca `next()` y `current()` del iterable a través de `VirtualCall` usando los slots conocidos del protocolo `Iterable`.

7.4. Lowering de lambdas y closures

El lowering de lambdas es uno de los procesos más interesantes del compilador. Al encontrar un nodo `HirExprKind::Lambda`:

1. Se identifican las variables capturadas (variables del entorno que se usan dentro de la lambda pero que no son parámetros suyos).
2. Se genera una función IR separada con kind `Lambda`, que recibe un parámetro implícito adicional: la closure misma.
3. En el cuerpo de esa función, los accesos a las variables capturadas se convierten en instrucciones `GetCapture` indexadas por posición.
4. En el sitio de creación de la lambda, se emite `MakeClosure` con la función generada y la lista de valores capturados actuales.

Esta estrategia separa limpiamente el cuerpo de la lambda de su entorno de captura, que es la representación estándar en compiladores modernos con soporte de funciones anónimas.

8. Backend LLVM y runtime

8.1. Crate `hulk-codegen-llvm`: emisión de LLVM IR

El crate `hulk-codegen-llvm` convierte un `IrProgram` en un módulo textual de LLVM IR (2190 líneas). La función de entrada es `emit_llvm(&IrProgram) -> Result<String, LlvmCodegenError>`.

El proceso de emisión tiene varias etapas:

1. Declarar las funciones externas del runtime (`declare void @hulk_print_number(double), etc.`).
2. Emitir las vtables de los tipos de usuario como constantes globales de tipo `[N x ptr]`.
3. Emitir las cadenas de texto estáticas como constantes globales de tipo `[N x i8]`.

4. Emitir las funciones del programa (globales, métodos, lambdas, entry).
5. Emitir una función `main()` C-compatible que llama a `hulk_entry()` y retorna su resultado.

8.2. Mapeo de tipos IR a LLVM

Tipo IR	Tipo LLVM	Comentario
Number	double	Aritmética y comparaciones en punto flotante.
Boolean	i1	Se convierte a i8 en fronteras con C si es necesario.
String	ptr	Puntero a <code>HulkString</code> del runtime.
Object y User(T)	ptr	Puntero a objeto con vtable al inicio.
Vector(T)	ptr	Puntero a <code>HulkVector</code> .
Iterable(T)	ptr	Objeto iterable con vtable (puede ser Vector o Range).
Functor	ptr	Puntero a <code>HulkClosure</code> .

La decisión de usar `ptr` opaco (LLVM opaque pointers) en lugar de tipos estructurados tipados (`%struct.HulkObject*`) simplifica la emisión de LLVM IR porque todos los objetos del runtime son punteros intercambiables. El backend no necesita conocer el layout interno de los objetos; eso es responsabilidad del runtime.

Sin embargo, esto generó un problema de compatibilidad: clang <16 requería la flag `-mllvm -opaque-pointers` para manejar el tipo `ptr`, mientras que clang 16+ lo quitó porque los punteros opacos ya son el comportamiento predeterminado. La solución fue detectar la versión de clang en tiempo de ejecución:

Listing 19: Detección dinámica de versión de clang

```
let needs_opaque_flag = process::Command::new("clang")
    .arg("--version")
    .output()
    .ok()
    .and_then(|o| String::from_utf8(o.stdout).ok())
    .and_then(|s| {
        s.split_whitespace()
            .skip_while(|t| *t != "version")
            .nth(1)
            .and_then(|v| v.split('.').next())
            .and_then(|maj| maj.parse::<u32>().ok())
    })
```

```
.map(|maj| maj < 16)
.unwrap_or(false);
```

Este fragmento aparece en `hulk-driver/src/main.rs` y también en `hulk-codegen-llvm/tests/backend_` para las pruebas de integración. La misma lógica en ambos sitios garantiza que tanto el driver como las pruebas sean compatibles con clang 14 (desarrollo local) y clang 17/18 (CI).

8.3. ABI con el runtime

El backend declara las siguientes funciones externas del runtime al inicio del módulo LLVM:

Listing 20: Declaraciones de ABI con el runtime (completas)

```
; Salida
declare void @hulk_print_number(double)
declare void @hulk_print_bool(i8)
declare void @hulk_print_string(ptr)
; Strings
declare ptr @hulk_string_concat(ptr, ptr)
declare ptr @hulk_string_concat_spaced(ptr, ptr)
declare i8 @hulk_string_equals(ptr, ptr)
declare ptr @hulk_string_from_number(double)
declare ptr @hulk_string_from_bool(i8)
declare i64 @hulk_string_len(ptr)
declare ptr @hulk_string_sub(ptr, i64, i64)
; Matemáticas estándar
declare double @hulk_sqrt(double)
declare double @hulk_sin(double)
declare double @hulk_cos(double)
declare double @hulk_exp(double)
declare double @hulk_log(double, double)
declare double @hulk_pow(double, double)
declare double @hulk_rand()
; Matemáticas extendidas
declare double @hulk_abs(double)
declare double @hulk_floor(double)
declare double @hulk_ceil(double)
declare double @hulk_round(double)
declare double @hulk_min(double, double)
declare double @hulk_max(double, double)
; Objetos y vtables
declare ptr @hulk_alloc_object(i64, i64, ptr)
declare ptr @hulk_object_method(ptr, i64)
declare double @hulk_object_get_number(ptr, i64)
declare void @hulk_object_set_number(ptr, i64, double)
declare i8 @hulk_object_is(ptr, i64)
declare ptr @hulk_object_as(ptr, i64)
```

```

; Vectores
declare ptr @hulk_range(double, double)
declare ptr @hulk_vector_new()
declare void @hulk_vector_push(ptr, i64)
declare i64 @hulk_vector_len(ptr)
declare i64 @hulk_vector_get(ptr, i64)
declare void @hulk_vector_set(ptr, i64, i64)
; Closures
declare ptr @hulk_closure_alloc(ptr, i64)
declare void @hulk_closure_set_capture(ptr, i64, ptr)
declare ptr @hulk_closure_get_capture(ptr, i64)
; Errores fatales
declare void @hulk_abort(ptr)

```

Cada categoría de operación del runtime tiene su propia familia de funciones con tipos concretos. Esto evita usar un mecanismo de boxing universal que requeriría más conversiones y aumentaría la complejidad del backend.

8.4. Crate runtime/: estructuras y modelo de objetos en C

El runtime está en `runtime/hulk_runtime.h` y `runtime/hulk_runtime.c`. Define las estructuras concretas que se crean en el heap durante la ejecución.

La estructura de vtable es el núcleo del modelo de objetos:

Listing 21: Estructura HulkVTable

```

typedef struct HulkVTable {
    int64_t          type_id;
    struct HulkVTable* parent;    // para traversal en is/as
    int64_t          method_count;
    void**           methods;     // array de punteros a funciones
    const char*      type_name;   // para mensajes de error en runtime
} HulkVTable;

```

En LLVM IR el tipo correspondiente es `%HulkVTable = type { i64, ptr, i64, ptr, ptr }`.

Una invariante crítica de diseño: **todos los objetos del runtime (objetos de usuario, vectores, rangos) deben comenzar con un `HulkVTable*` como primer campo**. Esto permite que `hulk_object_method` pueda obtener el slot correcto de cualquier objeto sin conocer su tipo concreto en tiempo de compilación.

Los objetos de usuario se asignan con `hulk_alloc_object(type_id, attr_count, vtable)`. Internamente, el runtime mantiene un array flexible de atributos representados como `int64_t` (64 bits), suficiente para almacenar doubles (por bits), punteros y valores booleanos. Los accessors diferenciados (`get_number/set_number`, `get_string/set_string`, etc.) hacen las conversiones necesarias.

Los vectores siguen la misma invariante:

Listing 22: Estructura HulkVector

```
typedef struct HulkVector {
    HulkVTable* vtable; // primer campo: slot 0 = next, slot 1 = current
    int64_t* data; // array dinámico de elementos como int64_t
    int64_t len;
    int64_t cap;
    int64_t cursor; // para iteración como Iterable
} HulkVector;
```

Los rangos también empiezan con HulkVTable* y son iterables directamente:

Listing 23: Estructura HulkRange como iterable

```
typedef struct HulkRange {
    HulkVTable* vtable; // slot 0 = next, slot 1 = current
    double current_val;
    double end_val;
} HulkRange;
```

Las closures usan un arreglo flexible de capturas (C99 flexible array member):

Listing 24: Estructura HulkClosure con capturas

```
typedef struct HulkClosure {
    void* fn_ptr; // puntero a la función lambda
    int64_t num_captures; // número de capturas
    void* captures[]; // flexible array: capturas como punteros opacos
} HulkClosure;
```

Toda captura se almacena como void*. En el lado LLVM, los valores no-puntero (doubles, bools) deben coercionarse a punteros antes de ser capturados, lo que es un área que requiere cuidado adicional.

8.5. Builtins matemáticos y métodos de String

Además de las funciones matemáticas básicas (`sqrt`, `sin`, `cos`, `exp`, `log`, `pow`, `rand`), el compilador expone seis builtins adicionales declarados en `hulk-sema/src/builtins.rs`:

Builtin	Aridad	Función runtime
<code>abs(x)</code>	1	<code>hulk_abs(double) ->double</code>
<code>floor(x)</code>	1	<code>hulk_floor(double) ->double</code>
<code>ceil(x)</code>	1	<code>hulk_ceil(double) ->double</code>
<code>round(x)</code>	1	<code>hulk_round(double) ->double</code>
<code>min(a,b)</code>	2	<code>hulk_min(double,double) ->double</code>

<code>max(a,b)</code>	2	<code>hulk_max(double,double) ->double</code>
-----------------------	---	--

Las llamadas a métodos sobre un receptor de tipo `String` se enrutan a funciones dedicadas del runtime desde `emit_string_method_call` en el backend:

- `s.length() / s.size()` → llama `@hulk_string_len(ptr)` → i64, convierte el resultado a double.
- `s.substring(start, end)` → convierte los argumentos `Number` a i64, llama `@hulk_string_sub(ptr, i64, i64)` → ptr.

8.6. Comprobaciones de seguridad en runtime

El compilador implementa comprobaciones de seguridad en tiempo de ejecución que abortan con un mensaje diagnóstico ante operaciones ilegales. La función central es:

Listing 25: Función de abort del runtime

```
void hulk_abort(const char* msg) {
    fputs("hulk runtime error: ", stderr);
    fputs(msg, stderr);
    fputc('\n', stderr);
    exit(1);
}
```

División y módulo por cero. El backend emite un guard antes de cada operación `/` o `%`:

Listing 26: Guard de división por cero generado por el backend

```
%divcheck = fcmp oeq double %rhs, 0.0
br i1 %divcheck, label %dz_err.N, label %dz_ok.N
dz_err.N:
    call void @hulk_abort(ptr @hulk_err_divzero)
    unreachable
dz_ok.N:
    %result = fdiv double %lhs, %rhs
```

Desreferencia nula en métodos. `hulk_object_method` verifica que el receptor no sea nulo antes de acceder a la vtable. Un receptor nulo emite “method call on null object” y termina.

Bounds de vtable. `hulk_object_method` verifica que `0 <= slot < method_count`.

Acceso a atributos en nulo. Las ocho funciones de acceso a atributos (`hulk_object_get_number`, `hulk_object_set_string`, etc.) comprueban que el puntero no sea nulo.

Índice de vector fuera de rango. `hulk_vector_get` y `hulk_vector_set` rechazan índices negativos e índices $\geq \text{len}$.

Fallo de cast. `hulk_object_as` devuelve nulo si el test de tipo falla.

8.7. Driver e integración con clang

El driver (`hulk-driver/src/main.rs`) implementa el modo de compilación completo y cinco modos de debug (`-ast`, `-check`, `-hir`, `-ir`, `-emit-llvm`).

En modo de compilación, el proceso es: `parse` → `analyze` → `lower` → `emit LLVM` → escribir `output.ll` → invocar `clang` con el runtime → eliminar `output.ll`. La búsqueda del runtime usa una estrategia de dos niveles: primero busca relativo al ejecutable (útil cuando se instala), y si no lo encuentra, relativo al directorio de trabajo (útil en desarrollo).

Los códigos de salida del driver están documentados: código 1 para error léxico, 2 para error sintáctico, 3 para error semántico, 4 para error de lowering, 5 para error de codegen LLVM, 6 para error al escribir el archivo IR, 7 para error al invocar `clang`. Esta granularidad facilita la automatización y la depuración.

8.8. Makefile

Listing 27: Objetivos del Makefile

```
build:
    cargo build --release -p hulk-driver
    cp target/release/hulkc ./hulk

test: build
    bash tests/hulk/run_tests.sh . tests/hulk

clean:
    cargo clean
    rm -f hulk output output.ll
```

9. Estrategia de pruebas

La suite de pruebas del compilador suma **529 tests unitarios e integración** (ejecutables con `cargo test -workspace`) más **115 programas HULK de extremo a extremo** distribuidos en carpetas por categoría. La filosofía es probar cada fase de forma aislada y luego verificar el pipeline completo con programas reales.

9.1. Pruebas unitarias por capa

hulk-lexgen. Los tests en `hulk-lexgen/tests/` verifican que la especificación declarativa de lexer genera un lexer correcto. Cubren: reconocimiento exacto de palabras clave y operadores, reglas de identificadores y números, strings con secuencias de escape, comentarios de línea, y rechazo de caracteres inválidos. Los tests de `normalize_spec` y `parse_spec` validan que la propia representación de la especificación se construye correctamente.

hulk-parsegen. Los tests en `hulk-parsegen/tests/` verifican el Pratt parser con una gramática mínima y con la gramática completa de HULK. Se comprueban: precedencia de los ocho niveles de operadores, asociatividad izquierda y derecha, operadores unarios prefijos (`-`, `!`), llamadas con argumentos, acceso por punto, indexación, agrupación por paréntesis, y manejo de errores para paréntesis no cerrado y RHS faltante.

hulk-frontend. Los tests en `hulk-frontend/tests/` ejercitan el `AstBuilder` sobre el CST producido por el parser. Verifican que los nodos del AST se construyen con los campos correctos: `ast_builder_expr.rs` cubre literales, binarios, unarios y llamadas; `control_flow.rs` cubre `if`, `while` y `for`; `functions_and_blocks.rs` cubre declaraciones de función, bloques y `let`; `types_frontend.rs` cubre `type`, `inherits` y protocolos.

hulk-sema. La fase semántica tiene 16 archivos de test con un total de más de 290 funciones de prueba. La cobertura se organiza por área:

- **Resolución básica** (`resolver_basic.rs`, `analysis.rs`): variables definidas y no definidas, shadowing léxico, scopes de `let`, parámetros de función, acceso a `self` dentro y fuera de tipos.
- **Chequeo de tipos básico** (`type_checker_basic.rs`, `sema_phase1/2/3.rs`): operandos de tipos incompatibles, condiciones no booleanas, retornos, asignaciones, aridad de llamadas.
- **Inferencia de tipos** (`function_param_inference.rs`, `method_param_inference.rs`, `type_constructor_param_inference.rs`, `inference_hardening.rs`): inferencia desde el cuerpo de la función, desde el sitio de llamada, inferencia interprocedural en cadenas, parámetros de métodos, parámetros de constructores de tipo. Los tests de *hardening* cubren casos límite como funciones sin ninguna llamada (error esperado), inferencia circular, y funciones polimórficas usadas con tipos distintos en llamadas distintas.
- **Protocolos y conformidad** (`protocol_conformance.rs`): tipos que conforman un protocolo, tipos que no conforman (método faltante, firma incorrecta), herencia de protocolo, uso de variables de tipo protocolo.
- **Métodos y despacho** (`method_calls.rs`): llamadas a métodos de tipos propios y heredados, override de métodos, uso de `base()`, despacho sobre variables de tipo protocolo.
- **Iterables y for** (`iterables.rs`): tipos que implementan el protocolo `Iterable`, uso de `for` sobre rangos y tipos de usuario, rechazo de tipos no iterables.
- **Vectores** (`vector_index.rs`): indexación válida e inválida, tipo del elemento inferido, asignación por índice.

- **Duplicados y errores de declaración** (`duplicates.rs`): funciones duplicadas, parámetros duplicados, atributos duplicados.
- **Pattern matching** (`match_patterns.rs`): wildcard, literales numéricos/booleanos/string, binding, patrón de tipo sin `as`, patrón de tipo con `as` (type narrowing), múltiples brazos, mezcla de tipos de patrón, y rechazo de match no exhaustivo.

hulk-lower. Los tests en `hulk-lower/tests/golden.rs` comparan la IR textual producida por el lowering contra archivos golden preestablecidos. Esto garantiza que cambios en el lowering no produzcan regresiones silenciosas en la estructura de la IR generada.

hulk-codegen-llvm. Los tests en `hulk-codegen-llvm/tests/` compilan programas HULK con `clang` y verifican la salida estándar exacta del binario resultante. Los seis archivos cubren:

- **backend_minimal.rs**: aritmética básica, booleanos, strings, concatenación, funciones, recursión, ciclos, builtins matemáticos, y un test de comportamiento negativo que verifica rechazo limpio de características no implementadas.
- **backend_objects_basic.rs**: creación de objetos, atributos, llamadas a métodos con `self`, constructores.
- **backend_inheritance_basic.rs**: herencia simple, override de métodos, acceso a atributos del padre.
- **backend_vtables.rs**: despacho polimórfico mediante vtable, variables de tipo base, correctitud del slot correcto ante overrides.
- **backend_type_ops.rs**: operadores `is` y `as`, cast de tipos, comportamiento en herencia multinivel.
- **backend_equality.rs**: comparación de igualdad sobre primitivos, strings y objetos.

9.2. Suite de programas HULK de extremo a extremo

La carpeta `tests/hulk/` contiene **115 programas** organizados en dos grupos.

Programas correctos (84 casos, `tests/hulk/ok/`). Cada programa tiene su archivo de salida esperada (`.expected`). El script `run_tests.sh` compila cada programa con `./hulk` y compara la salida del binario con la esperada. Las categorías son:

Categoría	Casos	Qué se valida
-----------	-------	---------------

<code>minimal</code>	20	Expresiones básicas, aritmética, strings, condicionales, bucles, funciones, recursión, <code>let/shadowing</code> , precedencia.
<code>types</code>	10	Anotaciones de tipo, inferencia desde cuerpo y llamada, builtins matemáticos, retornos tipados.
<code>oop</code>	10	Clases, herencia, overrides, <code>self</code> , <code>base()</code> , polimorfismo, mutación de atributos, vectores en OOP.
<code>arrays</code>	8	Vectores literales, inicializadores funcionales, 2D, mutación por índice, paso como argumento.
<code>interfaces</code>	6	Protocolos estructurales, polimorfismo sin herencia, herencia de interfaz.
<code>lambdas</code>	6	Lambdas simples, closures que capturan variables, funciones de orden superior, currying.
<code>generators</code>	6	Iterables de usuario con <code>next/current</code> , uso en <code>for</code> , generadores de secuencias.
<code>macros</code>	8	Declaraciones <code>define</code> , higiene de parámetros, <code>define</code> recursivo.
<code>extras</code>	10	<code>for</code> sobre rangos, bucles complejos, contadores, funciones con <code>for</code> .

Programas con error esperado (31 casos, `tests/hulk/errors/`). Cada programa debe ser *rechazado* por el compilador con el código de salida correcto. El script verifica el código de salida y la presencia de un mensaje de error en `stderr`. Las categorías son:

- **Léxicos (6):** carácter inválido, string no terminado, secuencia de escape inválida, carácter especial no permitido.
- **Sintácticos (10):** paréntesis sobrante, asignación inválida, `for` sin `in`, `let` sin `in`, punto y coma faltante, cuerpo de tipo faltante, llave no cerrada.
- **Semánticos (15):** variable no declarada, función no declarada, método no definido, aridad incorrecta, tipo incompatible, herencia circular, herencia de tipo no definido, tipo no iterable, condición no booleana, acceso a campo privado externo, redeclaración de función.

Esta cobertura de casos negativos es importante: valida que el compilador no solo acepta lo correcto, sino que rechaza lo incorrecto con el diagnóstico adecuado.

9.3. CI

El workflow de GitHub Actions ejecuta la suite completa de 529 tests unitarios bajo una instalación limpia de Rust estable, garantizando que no haya dependencias implícitas del entorno de desarrollo. La pipeline también verifica que el binario `hulkc` compile y que los

programas de la suite de extremo a extremo produzcan la salida esperada. Cualquier regresión en cualquier capa —lexer, parser, semántica, IR, backend— rompe la pipeline antes de que llegue a una rama protegida.

10. Extensiones del lenguaje

El compilador implementa nueve extensiones sobre el núcleo mínimo de HULK. Cada una atraviesa múltiples fases del compilador: no son azúcar sintáctica superficial, sino características que requirieron cambios coordinados en lexer, parser, AST, semántica, IR, lowering, backend y/o runtime. A continuación se documentan en profundidad, con su motivación, su alcance de implementación y ejemplos tomados directamente de la suite de pruebas.

10.1. Extensión 1: inferencia de tipos sin anotaciones

10.1.1. Motivación

En el núcleo de HULK, las funciones pueden o no llevar anotaciones de tipo en sus parámetros y retorno. Sin inferencia, todos los parámetros sin anotación quedarían con tipo `Unknown` y el compilador rechazaría el programa. La inferencia automática permite escribir funciones genéricas sin sacrificar seguridad:

Listing 28: Inferencia de tipos desde el cuerpo y el sitio de llamada

```
function square(x) {      // x no tiene anotacion
    x * x;                // la multiplicacion implica que x es Number
}

function clamp(val, lo, hi) {
    min_val(max_val(val, lo), hi); // inferencia interprocedural
}

if (square(5) == 25) print("ok") else print("fail");
if (clamp(10, 0, 5) == 5) print("ok") else print("fail");
```

10.1.2. Cómo se implementó

La inferencia opera en el `HirBuilder` (crate `hulk-sema`) mediante un algoritmo de punto fijo iterativo:

1. **Pasada de cuerpo:** se analiza el cuerpo de cada función. Si un parámetro aparece en una operación aritmética, se infiere `Number`; si aparece en una concatenación de strings, se infiere `String`; si aparece en un `!`, se infiere `Boolean`.

2. **Pasada de sitio de llamada:** se analizan los argumentos en cada invocación. Si se llama `f(42)`, se refina el primer parámetro de `f` a `Number`.
3. **Iteración hasta punto fijo:** si alguna firma cambió en la pasada, se repite. Esto propaga tipos a través de la cadena de llamadas (inferencia interprocedural).
4. **Pasada de error final:** solo cuando no hay más cambios se reportan parámetros que permanecen con tipo `Unknown` (error `CannotInferParameterType`).

La misma lógica se aplica a métodos y a parámetros de constructores de tipos. Las pruebas de `hulk-sema/tests/function_param_inference.rs`, `method_param_inference.rs` y `type_constructor_param_inference.rs` verifican estos casos. Un caso que falla correctamente es `function id(x) =>x`; sin ninguna llamada: el parámetro queda sin restringir y el compilador emite un error diagnóstico específico.

10.1.3. Por qué mejora el sistema

Sin esta extensión, escribir una función auxiliar simple como `max`, `clamp` o `sum` requeriría anotar explícitamente todos los parámetros. La inferencia reduce el ruido tipográfico sin perder las garantías del sistema de tipos: el compilador sigue validando usos inconsistentes y los diagnósticos de error son precisos.

10.2. Extensión 2: protocolos e interfaces estructurales

10.2.1. Motivación

HULK es un lenguaje con herencia nominal (las relaciones son explícitas). Sin protocolos, una función solo puede aceptar un tipo concreto o su jerarquía de herencia. Los protocolos añaden una segunda dimensión: la tipificación estructural. Una función puede declarar que acepta cualquier tipo que implemente ciertos métodos, sin importar la jerarquía de herencia.

Listing 29: Polimorfismo estructural con interfaces

```
interface Shape {
    area(): Number;
    name(): String;
}

type Square(s: Number) {
    side: Number = s;
    area(): Number { self.side * self.side; }
    name(): String { "square"; }
}

type Triangle(b: Number, h: Number) {
    base: Number = b;
```



```

    height: Number = h;
    area(): Number { self.base * self.height / 2; }
    name(): String { "triangle"; }
}

let sq: Shape = new Square(4) in
  if (sq.area() == 16) print("ok") else print("fail");

let tr: Shape = new Triangle(6, 4) in
  if (tr.area() == 12) print("ok") else print("fail");

```

`Square` y `Triangle` no heredan entre sí ni de una clase base común. Ambas conforman `Shape` porque implementan sus métodos. Una variable de tipo `Shape` puede apuntar a cualquiera de los dos.

10.2.2. Cómo se implementó

La implementación atraviesa el compilador completo:

- **Lexer/parser:** se reconocen las palabras clave `protocol`, `interface` (alias de `protocol`) y `extends`.
- **AST:** nodo `ProtocolDecl` con lista de firmas de métodos requeridos.
- **TypeRegistry:** la construcción del registro valida la conformidad de cada tipo de usuario contra cada protocolo que aparezca como anotación de parámetro o variable. La validación comprueba que la aridad y los tipos sean compatibles.
- **HIR:** cuando se llama un método sobre un receptor de tipo protocolo, el `DispatchKind` es `Virtual`, lo que garantiza despacho dinámico por `vtable` en tiempo de ejecución.
- **IR y backend:** el `VirtualCall` busca el slot del método en la `vtable` del objeto concreto; el protocolo solo define que ese slot existe, no qué función apunta.

El protocolo `Iterable` ocupa un lugar especial: se registra automáticamente con dos métodos requeridos (`next(): Boolean` y `current(): Object`). Esto permite que cualquier tipo que implemente esos dos métodos sea consumible en un `for`, sin que el usuario tenga que declarar explícitamente que lo implementa.

10.2.3. Por qué mejora el sistema

Los protocolos e interfaces hacen posible el polimorfismo sin herencia forzada. Sin ellos, para hacer que `Square` y `Triangle` sean intercambiables habría que hacerlos heredar de una clase base abstracta, lo que contamina el diseño de la jerarquía de tipos. Con protocolos, el contrato es puramente de comportamiento. Esto también desacopla los módulos: una función puede

declarar que solo necesita un `Printable` sin importarle si ese objeto es un `Point`, una `Person` o cualquier otro tipo futuro.

10.3. Extensión 3: arreglos indexados con inicialización funcional

10.3.1. Motivación

El lenguaje base no tiene colecciones de acceso directo por índice. La adición de arreglos permite representar tablas, matrices y secuencias con acceso en $O(1)$, lo que abre la puerta a algoritmos numéricos, procesamiento de datos y estructuras de mayor complejidad.

10.3.2. Tres formas de creación

Listing 30: Arreglo simple, con inicializador y bidimensional

```
// Arreglo simple: asignacion manual de elementos
let a: Number[] = new Number[5] in {
  a[0] := 10; a[1] := 20; a[2] := 30;
  if (a[2] == 30) print("ok") else print("fail");
};

// Inicializador funcional: nuevo Number[5]{ i -> i * 2 }
let a: Number[] = new Number[5]{ i -> i * 2 } in
  if (a[3] == 6) print("ok") else print("fail");

// Bidimensional: vector de vectores
let matrix: Number[][] = new Number[][3] in {
  let i = 0 in while (i < 3) {
    matrix[i] := new Number[3];
    i := i + 1;
  };
  if (matrix[1][2] == 0) print("ok") else print("fail");
};
```

10.3.3. Cómo se implementó

- **Pratt parser:** el prefijo `new` seguido de un identificador y `[` activa la rama de arreglo. El parser detecta si hay `][` (para `new T[][n]`), luego parsea la expresión de tamaño, y opcionalmente un bloque `{ var ->body }` como inicializador.
- **AST:** `Expr::NewVector{element_type, size, init}` donde `init` es opcional y contiene el nombre de la variable de índice y el cuerpo del inicializador.
- **HIR:** `HirExprKind::VectorNew` con `element_type`, `size` y `HirVectorNewInit` opcional.

- **IR:** instrucción `NewVector{dst, elements}` para vectores literales; para `new T[n]`, el lowering genera un bucle que invoca `VectorPush` para cada posición.
- **Runtime:** `HulkVector` con capacidad dinámica. La estructura mantiene longitud, capacidad y cursor de iteración. Los elementos se almacenan como `int64_t`, lo que permite representar tanto enteros y doubles (mediante reinterpretación de bits) como punteros a objetos.

10.3.4. Por qué mejora el sistema

Los arreglos son la estructura de datos más básica en cualquier lenguaje de propósito general. Sin ellos, implementar sumas de secuencias, búsqueda, ordenación o cualquier algoritmo que requiera memoria indexada es imposible. El inicializador funcional (`{ i ->expr }`) es especialmente valioso porque permite crear arreglos con valores calculados en una sola expresión, sin necesidad de bucles explícitos de inicialización.

10.4. Extensión 4: generadores e iterables de usuario

10.4.1. Motivación

El protocolo `Iterable` de HULK permite que los usuarios definan fuentes de datos perezosas (lazy), que producen elementos uno a uno bajo demanda. Esto es más expresivo que los arreglos: un generador puede ser infinito, o puede producir elementos calculados sin almacenarlos todos en memoria.

Listing 31: Generador de números pares como tipo usuario

```
type Evens(limit: Number) {
  current_val: Number = -2;
  max_val: Number = limit;

  next(): Boolean {
    self.current_val := self.current_val + 2;
    self.current_val <= self.max_val;
  }

  current(): Number { self.current_val; }
}

let sum = 0 in {
  for (x in new Evens(10)) {
    sum := sum + x;
  };
  if (sum == 30) print("ok") else print("fail");
};
```

10.4.2. Cómo se implementó

- **TypeRegistry:** valida que un tipo con anotación `T*` en parámetros o variables implemente los métodos `next(): Boolean` y `current(): Object` (o un subtipo de `Object`).
- **HIR:** el nodo `HirExprKind::For` contiene el tipo del iterable como `HirParam`; el `DispatchKind` de las llamadas a `next()` y `current()` es `Virtual`, lo que permite que cualquier tipo conforme funcione.
- **Lowering:** un `for (x in gen)` se baja a un bucle con `VirtualCall` a `next()` (resultado booleano), un `Branch` de salida si el resultado es falso, y un `VirtualCall` a `current()` para obtener el valor de cada iteración.
- **Builtin range:** se implementa en el runtime como `HulkRange`. La función `hulk_range(start, end)` asigna un `HulkRange` con `vtable` propia, cuyos slots 0 y 1 apuntan a `hulk_range_next` y `hulk_range_current`. Así, `range` es simplemente otro iterable, sin tratamiento especial en el compilador.

10.4.3. Por qué mejora el sistema

El patrón de generador desacopla la lógica de producción de datos de la lógica de consumo. Un `for` sobre un iterable funciona igualmente con un `range` builtin, con un `HulkVector`, o con cualquier tipo de usuario que implemente el protocolo. Esto hace que el `for` sea verdaderamente polimórfico y extensible: un usuario puede escribir un generador de Fibonacci, un generador de permutaciones, un cursor de base de datos, y todos funcionan con la misma sintaxis de `for`.

10.5. Extensión 5: lambdas, closures y funciones de orden superior

10.5.1. Motivación

Las funciones de primera clase permiten pasar comportamiento como dato. Sin lambdas, implementar funciones como `map`, `filter` o `reduce` requeriría crear un tipo de objeto para cada operación. Con lambdas, la abstracción es directa y el código es más conciso.

Listing 32: Lambda como argumento y clausura sobre el entorno léxico

```
// Clausura: captura 'base' del entorno externo
let base: Number = 10 in
let add_base: (Number) -> Number = function (x: Number): Number -> x + base in {
    if (add_base(5) == 15) print("ok") else print("fail");
};

// Funcion de orden superior
function map_range(lo: Number, hi: Number, f: (Number) -> Number): Number {
    let sum = 0 in {
```

```

        for (i in range(lo, hi)) { sum := sum + f(i); };
        sum;
    };
}

if (map_range(1, 4, function (x: Number): Number -> x * x) == 14) print("ok") else
    print("fail");

// Currying: retornar una lambda desde una funcion
function make_adder(n: Number): (Number) -> Number {
    function (x: Number): Number -> x + n;
}

let add5 = make_adder(5) in
    if (add5(3) == 8) print("ok") else print("fail");

```

10.5.2. Cómo se implementó

La implementación de lambdas es uno de los procesos más complejos del compilador porque involucra análisis de captura, generación de código fuera del flujo normal y representación de closures en memoria.

- **Pratt parser:** la función `is_lambda_start()` hace un scan sin consumir tokens para detectar si la expresión entre paréntesis forma una lista de parámetros seguida de `=>` (lambda short-form) o de `function(params) ->body` (long-form). Esto es necesario porque `(x + 1)` y `(x: Number) =>x + 1` empiezan igual pero son construcciones completamente distintas.
- **HIR:** `HirExprKind::Lambda` con parámetros tipados, tipo de retorno resuelto y cuerpo HIR.
- **Lowering:** al encontrar una lambda, el lowering (a) identifica las variables del entorno que se usan en el cuerpo y no son parámetros de la lambda (capturas), (b) genera una función IR separada de kind `Lambda` que recibe la closure como contexto implícito, (c) reemplaza los accesos a variables capturadas por instrucciones `GetCapture{idx, ty}`, y (d) en el sitio de creación emite `MakeClosure{function, captures}` con los valores actuales de las variables capturadas.
- **Runtime:** `HulkClosure` con puntero a función y arreglo flexible de capturas. El backend emite `hulk_closure_alloc(fn_ptr, num_captures)` y una llamada a `hulk_closure_set_cap` por cada variable capturada.

10.5.3. Por qué mejora el sistema

Las lambdas transforman a HULK en un lenguaje que puede expresar patrones de programación funcional. La combinación de `for` sobre iterables y funciones de orden superior permite

escribir transformaciones de datos expresivas. El patrón `make_adder` (función que retorna una lambda que captura un parámetro del entorno) es la base de la programación en estilo currying, lo que mejora significativamente la expresividad del lenguaje.

10.6. Extensión 6: define como definición de expresión única

10.6.1. Motivación

En HULK extendido, `define` introduce una función cuyo cuerpo es una sola expresión, sin necesidad de llaves. Es el equivalente a una *expression-bodied function* o una definición de macro de sustitución directa. Para el usuario, `define` comunica que la función es simple, pura y preferiblemente sin efectos secundarios.

Listing 33: Equivalencia entre `define` y `function`

```
define double(x: Number): Number -> x * 2;

function double_fn(x: Number): Number { x * 2; }

if (double(5) == 10) print("ok") else print("fail");
if (double_fn(5) == 10) print("ok") else print("fail");
// ambas producen el mismo resultado
```

10.6.2. Cómo se implementó

- **Lexer:** se añade `define` como palabra clave reservada.
- **Parser:** una declaración que comienza con `define` sigue la misma gramática que `function` pero con `->` como separador en lugar de `=>`, y no acepta cuerpos con llaves.
- **AST:** `FunctionDecl` con el campo `is_macro`: `bool` en `true`.
- **Semántica:** el `HirBuilder` procesa las declaraciones `define` de la misma manera que las funciones, con la misma inferencia de tipos. La distinción queda en el campo `is_macro` por si el backend o un optimizador futuro quieren tratarlas de forma diferente (e.g., inlining automático).

Los 8 tests de la categoría `macros` cubren: definición simple, encadenamiento de múltiples `define`, condicionales dentro del cuerpo, higiene (que la variable del parámetro no escapa al ámbito externo), bucles dentro del cuerpo, `define` anidados y comparación directa con la versión `function` equivalente.

10.6.3. Por qué mejora el sistema

`define` es un marcador de intención. Ayuda al lector del código a distinguir rápidamente funciones de una expresión (preferiblemente puras) de funciones con cuerpos complejos de varias instrucciones. También abre la puerta a una optimización futura: si el compilador sabe que una función es una `define`, puede hacer inlining agresivo con mayor confianza de que no habrá efectos secundarios inesperados.

10.7. Extensión 7: for sobre iterables y el builtin range

10.7.1. Motivación

El bucle `while` obliga al programador a gestionar manualmente la variable de iteración, el avance y la condición de parada. Esto es verboso y propenso a errores (off-by-one, olvido de incremento). El `for` sobre iterables abstrae completamente ese patrón y garantiza que el protocolo se respete.

Listing 34: `for` sobre `range` como alternativa a `while` manual

```
// Con while (verboso, propenso a errores off-by-one)
let i = 0, sum = 0 in while (i < 5) {
    sum := sum + i;
    i := i + 1;
};

// Con for sobre range (declarativo, sin gestion manual)
let sum = 0 in {
    for (x in range(0, 5)) {
        sum := sum + x;
    };
    if (sum == 10) print("ok") else print("fail");
};
```

10.7.2. Comportamiento de range

El builtin `range(lo, hi)` produce los valores `lo`, `lo+1`, ..., `hi-1` (extremo superior exclusivo). La prueba `range_sum.hulk` confirma: `range(1, 6)` produce 5 elementos (1, 2, 3, 4, 5) con suma 15. Esto sigue la convención de Python/Rust, que es la más intuitiva para bucles de longitud conocida.

10.7.3. Cómo se implementó

- **Parser:** `for (var in expr) body` es una expresión completa en el Pratt parser, con la misma prioridad que `while`. El cuerpo puede ser cualquier expresión, incluyendo bloques.

- **HIR:** `HirExprKind::For{var, iterable, body}` donde `var` incluye su `SymbolId` único.
- **Lowering:** se genera el siguiente patrón IR:
 1. llamada `VirtualCall` a `next()` del iterable en slot 0;
 2. `Branch` sobre el resultado booleano;
 3. si verdadero, `VirtualCall` a `current()` en slot 1, asignación a la variable del bucle, ejecución del cuerpo, salto a la comprobación;
 4. si falso, salida del bucle.
- **Runtime:** `HulkRange` implementa `next()` y `current()` con `vtable` propia. La función `hulk_range(start, end)` asigna la estructura y configura `current_val = start - 1` para que el primer `next()` produzca exactamente `start`.

10.7.4. Por qué mejora el sistema

El `for` unifica tres patrones de uso: iterar un rango numérico (con `range`), iterar un arreglo (con su `vtable` de iterable), e iterar cualquier generador de usuario. El usuario no necesita aprender ninguna API diferente para cada caso. Además, el cuerpo del `for` es una expresión, lo que es consistente con la filosofía de HULK de que todo produce un valor.

10.8. Extensión 8: visibilidad protegida de atributos en jerarquías de herencia

10.8.1. Contexto: lo que dice la especificación

La especificación de HULK (p.38) establece: “*Attributes are always private... not even inheritors can access parent attributes directly.*” Interpretada de forma estricta, esta regla implicaría que un tipo hijo no puede usar `self.attr` si `attr` fue declarado en el padre.

Sin embargo, nuestro compilador implementa una semántica más permisiva dentro de la jerarquía de herencia, y lo hace de forma deliberada, con una lógica técnica precisa y con casos de prueba que lo validan. Esta sección documenta esa decisión: qué se hizo, cómo funciona en el código y por qué es una extensión que mejora el lenguaje.

10.8.2. Qué se implementó: visibilidad protegida

El modelo adoptado distingue dos tipos de acceso a atributos:

1. **Acceso interno a través de `self`** (dentro de un método propio o heredado): **permitido para toda la jerarquía**, incluyendo los atributos declarados en tipos padre.

2. **Acceso externo desde código ajeno a la jerarquía** (desde main o desde un tipo sin relación de herencia): **bloqueado** con error `AttributeIsPrivate`.

Esto equivale exactamente a la semántica `protected` de Java o C++: el atributo no es público, pero sí es accesible para la clase que lo declara y para todas sus subclases.

10.8.3. Evidencia directa en los tests

Los siguientes casos de prueba ilustran exactamente esta distinción:

Listing 35: `method_override.hulk`: `FancyPrinter` accede a `self.pfx` declarado en `Printer`

```
type Printer(prefix: String) {
  pfx: String = prefix;
  format(msg: String): String { self.pfx @ msg; }
}

type FancyPrinter(prefix: String, suffix: String) inherits Printer(prefix) {
  sfx: String = suffix;
  // FancyPrinter sobrescribe format() y accede a self.pfx (declarado en Printer)
  format(msg: String): String { self.pfx @ msg @ self.sfx; }
}

let fp = new FancyPrinter(">> ", " <<") in
  if (fp.format("hello") == ">> hello <<") print("ok") else print("fail");
```

Listing 36: `interface_inherit_compat.hulk`: `Button` accede a `self.label` declarado en `Widget`

```
type Widget(label: String) {
  label: String = label;
  draw(): String { "widget:" @ self.label; }
}

type Button(label: String) inherits Widget(label) {
  // Button sobrescribe draw() y accede a self.label (declarado en Widget)
  draw(): String { "button:" @ self.label; }
}
```

En ambos casos, el tipo hijo accede en su método overrideado a un atributo que fue declarado en el padre. Ambos tests están en la carpeta `tests/hulk/ok/`, lo que confirma que el compilador los acepta como programas válidos.

Por contraste, el test de error `tests/hulk/errors/semantic/wrong_field_access.hulk` contiene acceso externo a un atributo de un tipo desde código no relacionado, y el compilador rechaza ese programa con `AttributeIsPrivate`.

10.8.4. Cómo funciona en el compilador: la lógica exacta

La clave está en dos puntos del código del HirBuilder.

Punto 1 — `lookup_attribute` **recorre toda la cadena de herencia**. La función en `hulk-sema/src/context.rs` (línea 349) está implementada así:

Listing 37: `lookup_attribute` recorre la cadena de herencia completa

```
pub fn lookup_attribute(&self, type_name: &str, attr_name: &str) -> Option<Type> {
    let mut current = Some(type_name);
    while let Some(name) = current {
        let Some(ti) = self.types.get(name) else { break };
        if let Some(ty) = ti.attributes.get(attr_name) {
            return Some(ty.clone()); // encontrado en este nivel o en un ancestro
        }
        current = ti.parent.as_deref(); // sube al padre
    }
    None // no existe en la jerarquia
}
```

Cuando se analiza `FancyPrinter.format()` y se encuentra `self.pfx`, el compilador llama `lookup_attribute("FancyPrinter", "pfx")`. La función empieza en `FancyPrinter`, no encuentra `pfx` allí (solo tiene `sfx`), sube a `Printer`, encuentra `pfx` y retorna su tipo. El acceso es válido.

Punto 2 — **La regla de privacidad solo bloquea accesos externos**. En `hir_builder.rs` (línea 1653), `analyze_member_access` tiene dos ramas:

Listing 38: Lógica de acceso: `self` vs externo

```
if is_self {
    // Acceso via self: se permite si el atributo existe en CUALQUIER nivel de
    // la jerarquia (lookup_attribute sube hasta encontrarlo o agotar la cadena)
    if let Some(attr_ty) = self.registry.lookup_attribute(&owner_type, member) {
        return /* acceso valido */;
    }
    self.errors.push(SemanticError::AttributeIsPrivate { ... });
} else {
    // Acceso externo (obj.attr desde otro contexto)
    // Solo se permite si obj tiene exactamente el mismo tipo que self.current_type
    let same_class = self.current_type.as_deref()
        .map(|ct| ct == type_name).unwrap_or(false);
    if same_class {
        if let Some(attr_ty) = self.registry.lookup_attribute(&type_name, member) {
            return /* mismo tipo: permitido */;
        }
    }
    self.errors.push(SemanticError::AttributeIsPrivate { ... });
}
```

La distinción es precisa: `self.pfx` desde un método de `FancyPrinter` usa la rama de `self`, que busca en toda la jerarquía. Un acceso como `fp.pfx` desde el bloque principal usa la rama externa, que falla porque el contexto actual no es un método de `FancyPrinter`.

El caso adicional de la rama externa (`same_class`) permite también patrones como `other.x` dentro de un método de la misma clase: útil para comparaciones entre instancias del mismo tipo (como en `tests/hulk/ok/oop/vector_math.hulk`, donde `dot(other: Vector)` accede a `other.x` dentro de un método de `Vector`).

10.8.5. Justificaciones de por qué esta extensión mejora el lenguaje

1. La herencia sin acceso a estado heredado es semánticamente incompleta. Si un tipo hijo hereda todos los atributos del padre a nivel de memoria (lo cual es inevitable: el constructor del padre los inicializa y el hijo los posee físicamente en su layout de objeto), pero no puede leerlos ni modificarlos, entonces hereda estado invisible. El hijo puede invocar los métodos del padre mediante `base()`, pero si quiere *extender* o *especializar* esos métodos sin delegarlos completamente, no puede hacerlo sin duplicar el estado.

En `method_override.hulk`, el propósito de `FancyPrinter` es exactamente ese: extender el comportamiento de `Printer.format()` añadiendo un sufijo. Para eso necesita el prefijo del padre. Sin acceso a `self.pfx`, la única alternativa sería:

Listing 39: Alternativa sin acceso protegido: duplicacion innecesaria de estado

```
type FancyPrinter(prefix: String, suffix: String) inherits Printer(prefix) {
  pfx_copy: String = prefix; // duplicar el atributo del padre
  sfx: String = suffix;
  format(msg: String): String { self.pfx_copy @ msg @ self.sfx; }
}
```

Esta alternativa es redundante y rompe la cohesión: `pfx_copy` y `pfx` tienen el mismo valor pero son dos atributos distintos, ocupan más memoria, y si el padre llegara a modificar `pfx`, el hijo no vería el cambio.

2. El layout de objetos ya incluye los atributos del padre. En la IR de Hulk, el layout de `FancyPrinter` hereda los atributos del padre con sus `AttrId` originales. El atributo `pfx` de `Printer` tiene `AttrId #0`; cuando el lowering genera código para `FancyPrinter`, ese atributo sigue siendo `AttrId #0` en el objeto. En tiempo de ejecución, la memoria del objeto `FancyPrinter` contiene `pfx` en la misma posición que lo haría un objeto `Printer`. La restricción de visibilidad sería puramente artificial: el atributo está físicamente ahí y el índice es conocido en tiempo de compilación.

Bloquear el acceso no daría ninguna garantía de seguridad adicional a nivel de memoria; solo agregaría una limitación de expresividad sin contrapartida técnica.

3. La especificación probablemente protege de accesos externos, no internos. La frase “*not even inheritors*” en la especificación de HULK aparece en el contexto de discutir

la encapsulación frente a código externo. El espíritu de esa regla es evitar que cualquier código ajeno a la jerarquía lea o modifique el estado interno de un objeto. Nuestro compilador cumple ese espíritu: desde código externo (fuera de la jerarquía), ningún atributo es accesible. La extensión solo abre el acceso a los propios métodos del tipo y de sus subtipos, que conceptualmente son los más capacitados para operar sobre ese estado.

Esta interpretación es la misma que adoptaron Java, C#, C++, Kotlin y Swift con sus modificadores `protected`. La decisión de HULK de no tener `protected` como palabra clave explícita no necesariamente implica que el acceso dentro de la jerarquía deba ser idéntico al acceso externo.

4. Los overrides significativos requieren acceso al estado que se especializa. El patrón de override más común en OOP es el de *extensión*: un hijo agrega comportamiento a un método existente, conservando parte de la lógica del padre. En HULK esto se puede hacer con `base()`, pero `base()` llama al método completo del padre. Si el hijo quiere cambiar cómo se usa el estado del padre en el método (no solo añadir algo antes o después), `base()` no es suficiente.

En `FancyPrinter.format()`, el hijo no quiere llamar a `base()(msg)` y luego añadir algo; quiere construir una cadena completamente diferente que incorpore `self.pfx` en una posición diferente. Ese patrón requiere acceso directo al atributo.

5. El acceso mismo-clase (cross-instance) habilita patrones algebraicos. La rama `same_class` del compilador permite además que un método acceda a los atributos de *otro objeto del mismo tipo* (no solo de `self`). Esto es necesario para patrones como el producto escalar:

Listing 40: `vector_math.hulk`: acceso cross-instance al mismo tipo

```
type Vector(x: Number, y: Number) {
  x: Number = x;
  y: Number = y;
  dot(other: Vector): Number { self.x * other.x + self.y * other.y; }
}
```

Sin acceso `other.x` y `other.y`, implementar el producto escalar, la suma de vectores o cualquier operación binaria sobre instancias del mismo tipo requeriría exponer getters públicos, lo que es más verboso y menos eficiente (una llamada de método por cada acceso en lugar de un acceso directo a memoria).

6. La privacidad total sin excepciones hace la herencia más costosa que la composición. Si los atributos del padre son estrictamente inaccesibles para el hijo, la herencia pierde una de sus ventajas principales: reutilizar y especializar estado. El hijo puede reutilizar los métodos del padre (via `base()` o herencia directa), pero para trabajar con el estado hereda tiene que ir a través de la interfaz de métodos del padre, que puede ser más limitada que el acceso directo. Esto hace que en muchos casos sea preferible la composición (tener un

atributo del tipo padre en lugar de heredar), lo cual va en contra de los patrones de diseño orientados a objetos que la herencia en HULK busca habilitar.

10.8.6. Regla final aplicada por el compilador

La regla que implementamos se puede enunciar de forma precisa:

- **Acceso desde self dentro de un método:** se permite el acceso a cualquier atributo existente en el tipo actual o en cualquiera de sus ancestros. Esto incluye los atributos declarados en el padre.
- **Acceso sobre otra instancia del mismo tipo** (`other.attr` dentro de un método donde `other: MismoTipo`): se permite si el atributo existe en ese tipo o en sus ancestros.
- **Acceso externo** (desde código fuera de la jerarquía, o sobre un objeto de tipo diferente): bloqueado con error diagnóstico `AttributeIsPrivate`.

Esta regla es más precisa, más permisiva dentro de la jerarquía y más restrictiva fuera de ella que la interpretación más literal de la especificación. La consideramos una extensión justificada que hace la herencia en HULK semánticamente completa.

10.9. Extensión 9: pattern matching

10.9.1. Motivación

El despacho de tipos en HULK sin pattern matching requiere encadenar expresiones `is/as`:

Listing 41: Despacho de tipos sin pattern matching: verboso y propenso a errores

```
let result =
  if (s is Circle)
    "circle: " @ (s as Circle).area()
  elif (s is Rectangle)
    "rect: " @ (s as Rectangle).area()
  else
    "other"
in print(result);
```

Este patrón tiene varios problemas: el programador repite el nombre del tipo dos veces (`is Circle` y luego `as Circle`), el cast puede fallar en tiempo de ejecución si se comete un error de copia, y el compilador no puede verificar si todos los casos relevantes están cubiertos. Con muchos subtipos, el código se vuelve difícil de leer y mantener.

El pattern matching resuelve los tres problemas: el tipo se nombra una sola vez, el narrowing es automático, y el compilador verifica la exhaustividad en tiempo de compilación.

10.9.2. Sintaxis

Listing 42: Sintaxis de match: scrutinee, patrones y cuerpos de brazo

```
type Shape() { area(): Number => 0; }
type Circle(r: Number) inherits Shape { area(): Number => 3 * r * r; }
type Rectangle(w: Number, h: Number) inherits Shape { area(): Number => w * h; }

let s: Shape = new Circle(5) in
print(match (s) {
    Circle as c    => "circle: " @ c.area(),
    Rectangle as r => "rect: "   @ r.area(),
    _              => "other"
});
// imprime: circle: 75
```

La estructura general es `match (expr) { patron => cuerpo, ... }`. Cada brazo produce un valor; el resultado de la expresión `match` es el valor del brazo que se ejecute. Los tipos de todos los brazos se unifican (tipo común más derivado) para determinar el tipo de retorno de la expresión completa.

10.9.3. Tipos de patrones

Patrón	Ejemplo	Comportamiento
Wildcard	<code>_</code>	coincide con cualquier valor; no introduce variable
Literal	<code>42</code> , <code>"hi"</code> , <code>true</code>	coincide si el valor es exactamente igual al literal
Binding	<code>n</code>	coincide siempre; liga el escrutinio a <code>n</code>
Tipo	<code>Circle</code>	coincide si <code>scrutinee is Circle</code>
Tipo + bind	<code>Circle as c</code>	coincide + estrecha el tipo: <code>c</code> tiene tipo <code>Circle</code> en el cuerpo

La disambiguación se hace en el parser: un identificador que comienza con mayúscula es un patrón de tipo (convención de HULK para nombres de tipo); uno que comienza con minúscula es un patrón de binding; el token especial `_` (WILDCARD) es el comodín.

10.9.4. Verificación de exhaustividad

El compilador exige que el último brazo sea un patrón que garantice cobertura total: `_` (wildcard) o un binding. Si ningún brazo cumple esa condición, se emite el error `NonExhaustiveMatch` en tiempo de compilación:

Listing 43: Match no exhaustivo: error de compilación

```
match (s) { Circle => 1, Rectangle => 2 };
// error: Non-exhaustive match: missing a wildcard or binding arm
```

Esto previene el caso de que el match llegue en tiempo de ejecución a un valor no contemplado, garantizando que siempre hay un brazo que se ejecuta.

10.9.5. Cómo se implementó

La implementación siguió una estrategia de *desazucarado completo en el HIR builder*, de modo que las fases posteriores (IR, lowering, backend LLVM) no requirieron ningún cambio.

- **Lexer:** se añadió el token `MATCH` y el token `WILDCARD` (`_`) al lexer de HULK.
- **Gramática LL(1):** se añadió la producción `OperatorExpr -> MATCH`; en los tres archivos de gramática (`hulk_control.gx`, `hulk_types.gx`, `hulk_full.gx`). Esta producción actúa como punto de entrada al Pratt parser cuando el lookahead es `MATCH`.
- **Pratt parser:** se añadieron los campos `match_kw` y `wildcard` a `PrattConfig` y los métodos `parse_match_subexpr` y `parse_match_pattern` al runtime del Pratt parser. El resultado es un nodo CST `MatchExpr` con hijos `Expr` (escrutinio) y `MatchArm*` (pares patrón/cuerpo).
- **AST:** se añadió la variante `Expr::Match { scrutinee, arms }` y los tipos `MatchArm`, `Pattern` (con variantes `Wildcard`, `Literal(LiteralPattern)`, `Binding(String)`, `TypePattern { name, bind }`) al módulo `hulk-frontend`.
- **Resolver:** `Expr::Match` introduce los bindings de cada brazo en un scope propio: un patrón de binding define una variable igual al escrutinio; un patrón de tipo con `as` define una variable del tipo estrechado.
- **Type checker:** infiere el tipo del escrutinio, verifica que el tipo indicado en patrones de tipo exista en el registro, y determina el tipo de retorno del match como el tipo común de todos los cuerpos mediante `unify_types` (LCA en la jerarquía de herencia).
- **HIR builder:** desazucara `Expr::Match` en:
 1. Una variable sintética `_match_scr_N` ligada al escrutinio mediante `HirExprKind::Let`.
 2. Una cadena de condicionales `HirExprKind::If`: la condición de cada brazo de tipo es un `HirExprKind::TypeTest`; la de un brazo literal es una igualdad `HirExprKind::Binary{op: Eq}`; el brazo `wildcard` o binding cierra la cadena como la rama `else`.
 3. El binding de tipo (`as varname`) se desazucara a un `HirExprKind::Let` adicional que contiene un `HirExprKind::TypeCast`.

El resultado es que la IR, el lowering y el backend LLVM procesan el match como si fuera una expresión `let/if` ordinaria, sin necesidad de conocer el concepto de “match”.

10.9.6. Por qué mejora el sistema

El pattern matching aporta tres mejoras concretas al lenguaje:

1. Seguridad en el despacho de tipos. El cast implícito en el brazo `Circle` `as c` está garantizado: si el brazo se ejecuta, el escrutinio es de tipo `Circle`. No hay posibilidad de un cast incorrecto. En el enfoque con `is/as` el programador puede escribir `is Circle` pero luego hacer `as Rectangle` por error tipográfico; el compilador no lo detectaría hasta el análisis semántico posterior, y algunos compiladores no lo detectarían en absoluto.

2. Exhaustividad verificada en compilación. La obligación de un brazo final wildcard o binding transforma un error de runtime (valor no contemplado → comportamiento indefinido) en un error de compilación. El programa que llega a ejecutarse tiene cobertura total garantizada.

3. Expresividad y legibilidad. Un match sobre cinco subtipos ocupa cinco líneas limpias en lugar de cinco bloques `if/elif` con cuatro referencias al tipo cada uno. El código que usa pattern matching es más fácil de leer, de extender (añadir un brazo) y de revisar en una pull request.

Desde el punto de vista del compilador, la implementación demostró la robustez del diseño por capas: una extensión de lenguaje significativa (con su propio análisis, representación y semántica) se integró completamente en las fases de frontend y semántica sin afectar en absoluto al backend.

11. Comparación con la planificación inicial

Fase	Estado alcanzado	Comentario
Lexer y parser	Completos	Generadores propios, LL(1) + Pratt parser.
AST	Amplio	Todos los constructos de HULK incluyendo extensiones.
Semántica	Avanzada	Inferencia iterativa, protocolos, scopes, HIR tipada.
IR y lowering	Avanzados	Cubre más de lo que el backend puede ejecutar completamente.
Runtime/ejecución	Parcial funcional	LLVM + runtime C; no VM propia.
Extensiones	Nueve implementadas	Inferencia, protocolos, vectores, generadores, lambdas, <code>define</code> , <code>for</code> , visibilidad protegida, pattern matching.
Documentación	Este documento	Informe técnico del equipo.

La diferencia más relevante respecto a la planificación: en lugar de implementar una VM

propia como camino principal, el proyecto adoptó LLVM IR + runtime C. Esta decisión fue coherente con la orientación del proyecto y aceleró la llegada a un compilador ejecutable.

12. Limitaciones

12.1. Runtime sin recolector de basura

El runtime usa un arena bump-allocator de 64 MB. La memoria asignada con `hulk_alloc_object`, `hulk_vector_new` y `hulk_closure_alloc` no se libera individualmente; toda la arena se descarta al terminar el proceso. Para una entrega académica con programas cortos esto es aceptable, pero no sería válido para un sistema de producción de larga duración.

Los elementos de vector se representan como `int64_t`, lo que requiere conversiones bit a bit para doubles y punteros. El acceso por índice fuera de rango es detectado en runtime y aborta con mensaje diagnóstico.

12.2. Cobertura del backend

El backend LLVM tiene cobertura completa para tipos primitivos, strings, funciones globales, recursión, ciclos, OOP con herencia, protocolos con despacho virtual, vectores indexados, rangos, lambdas y closures simples. Las closures que capturan valores no-puntero (doubles almacenados directamente, no como objeto) requieren una coerción extra que está implementada pero puede generar comportamiento incorrecto en casos de captura de valores numéricos mutados después de la creación de la closure.

12.3. Macros

El sistema de `define` no es un sistema de macros higiénicas completo. No debe presentarse como tal en la defensa.

13. Instrucciones de uso

13.1. Compilación del proyecto

Listing 44: Construcción del compilador

```
make build
# equivale a:
cargo build --release -p hulk-driver
cp target/release/hulkc ./hulk
```

13.2. Uso del compilador

Listing 45: Modos de uso del driver

```
./hulk programa.hulk          # compila a ./output (requiere clang)
./output                      # ejecuta el binario
./hulk programa.hulk --ast     # imprime el AST
./hulk programa.hulk --check   # ejecuta el chequeo semántico
./hulk programa.hulk --hir     # imprime la HIR tipada
./hulk programa.hulk --ir      # imprime la Hulk IR
./hulk programa.hulk --emit-llvm # imprime el LLVM IR generado
```

13.3. Requisitos

- Rust estable (1.70+) con Cargo
- clang (versión 14 a 18 soportada; la detección automática de versión maneja la flag `-opaque-pointers`)
- libm (incluida en sistemas Linux/macOS)

14. Conclusiones

El compilador desarrollado presenta una arquitectura sólida y modular para un proyecto académico de compilación. La separación en ocho crates Rust, el uso de generadores propios de lexer y parser, el Pratt parser para expresiones, la HIR tipada como contrato entre fases, la IR propia con un conjunto de instrucciones completo, y el backend LLVM con ABI explícito con el runtime muestran una comprensión clara del diseño de un compilador moderno.

Las decisiones técnicas más relevantes del equipo fueron: usar `LexerSpec` declarativa en lugar de un lexer manual; combinar LL(1) con Pratt para el parser; implementar inferencia iterativa en la semántica; diseñar tipos fuertes para todos los identificadores de IR; separar `IrValue` de `IrPlace` para hacer ilegales las asignaciones inválidas; e incluir `capture_types` en `IrTypeRef::Functor` para contratos correctos con el backend.

Las limitaciones identificadas (runtime sin GC, backend con cobertura parcial, macros iniciales) son honestas y forman parte de una arquitectura incremental en la que las fronteras de completitud están bien identificadas. El mayor valor técnico está en la claridad del pipeline y en los contratos internos entre AST, HIR, IR, LLVM y runtime.

A. Checklist final de entrega

Elemento	Estado	Prioridad
cargo test -workspace -all-targets -locked pasa	Cumplido	Alta
CI de GitHub Actions pasa	Cumplido	Alta
Driver compila a ./output con ejemplos básicos	Cumplido	Alta
Dumps de AST, HIR, IR y LLVM funcionan	Cumplido	Alta
Documentar subconjunto ejecutable real	Este informe	Alta
Builtins matemáticos extendidos (abs, floor, ceil, round, min, max)	Cumplido	Media
Métodos de String (length/size, substring)	Cumplido	Media
Ubicación en errores semánticos (Undefined-Variable/Function/Method)	Cumplido	Media
Guards de división/módulo por cero	Cumplido	Alta
Comprobación de bounds en vectores	Cumplido	Media
Comprobación de nulos en métodos y atributos	Cumplido	Media
README con comandos de uso y ejemplos	Cumplido	Baja
Pattern matching (match/_)	Cumplido	Media
Estrategia de memoria / garbage collector	Futuro	Baja

B. Programa demostrativo sugerido

Listing 46: Programa que recorre el pipeline completo sin usar partes frágiles

```
function square(x: Number): Number => x * x;

type Point(x: Number, y: Number) {
  x: Number = x;
  y: Number = y;

  norm2(): Number => square(self.x) + square(self.y);
}

type Circle(cx: Number, cy: Number, r: Number) inherits Point(cx, cy) {
  r: Number = r;

  area(): Number => PI * square(self.r);
}

let p = new Point(3, 4) in {
  print("norm2:");
  print(p.norm2());
};
```

```
let c = new Circle(0, 0, 5) in {  
    print("area:");  
    print(c.area());  
};
```

Este programa ejercita: funciones globales, tipos, atributos, métodos con **self**, herencia simple, construcción de objetos, llamadas a métodos, strings, números y el builtin **PI**.